

PORTADA

Nombre Alumno / DNI	Oscar Garcia Panizo / 44440360J
Título del Programa	3ºPD CYBER SECURITY / COMPUTER SCIENCE
Nº Unidad y Título	UNIT 25. APPLIED MACHINE LEARNING
Año académico	2025/2026
Profesor de la unidad	Rabindranath Andujar
Título del Assignment	AB- Assignment Brief
Día de emisión	27/9/2025
Día de entrega	19/01/2026
Nombre IV y fecha	
Declaración del estudiante	Certifico que la presentación del assignment es completamente mi propio trabajo y entiendo completamente las consecuencias del plagio. Entiendo que hacer una declaración falsa es una forma de mala práctica. Fecha: 19/01/2026  Firma del alumno:

Plagio

El plagio es una forma particular de hacer trampa. El plagio debe evitarse a toda costa y los alumnos que infrinjan las reglas, aunque sea inocentemente, pueden ser sancionados. Es su responsabilidad asegurarse de comprender las prácticas de referencia correctas. Como alumno de nivel universitario, se espera que utilice las referencias adecuadas en todo momento y mantenga notas cuidadosamente detalladas de todas sus fuentes de materiales para el material que ha utilizado en su trabajo, incluido cualquier material descargado de Internet. Consulte al profesor de la unidad correspondiente o al tutor del curso si necesita más consejos.

Índice

1. Introducción
2. Objetivos
3. Marco teórico
4. Desarrollo del proyecto
 - 4.1 Dataset y preparación de datos
 - 4.2 EDA
 - 4.3 Modelado: algoritmos y comparativas
 - 4.4 Entrenamiento, métricas y evaluación
 - 4.5 Arquitectura y despliegue
 - 4.6 Comunicación cliente–servidor y ciclo de feedback
 - 4.7 Incidencias y resolución
5. Conclusiones
6. Evaluación crítica
7. Bibliografía

1. Introducción

1.1 Contexto del problema

El proyecto aborda un problema de **clasificación visual**: determinar el **estado de maduración/deterioro de un plátano** a partir de una imagen. La motivación es crear una herramienta rápida y accesible que convierta una señal visual (imagen) en una **predicción automática**.

1.2 Usuario final y valor de la predicción

Usuario final: aunque inicialmente este proyecto empezó como una broma tiene bastantes aplicaciones como separar plátanos en un estado de los podridos en cadenas de montaje, ayuda a personas ciegas para facilitarles hacer la compra o incluso con pequeños cambios poder detectar una epidemia de una enfermedad en plantaciones de plátanos de forma rápida

Valor: permitir una decisión rápida: consumir, esperar, descartar o no comprar. El valor aumenta si la predicción se entrega en tiempo real y con una métrica de confianza, evitando decisiones basadas en estimaciones subjetivas.

1.3 Hipótesis inicial

Hipótesis: un modelo de clasificación entrenado con imágenes etiquetadas puede alcanzar una precisión alta distinguiendo las 4 clases principales, aunque existan confusiones en casos limítrofes.

2. Objetivos

2.1 Objetivo general

Desarrollar y desplegar una aplicación web que, mediante cámara, detecte un plátano y **clasifique su estado** con un modelo de aprendizaje automático.

2.2 Objetivos específicos

- Entrenar un clasificador multiclase con 4 categorías: pasado, perfecto, podrido, verde.
- Evaluar el modelo con métricas cuantitativas (acierto, precisión, recall, F1 y matriz de confusión).
- Implementar una web con inferencia en cliente usando **ONNX Runtime Web**.

- Mejorar la robustez usando un pipeline de dos pasos: **detección (banana)** → **recorte** → **clasificación**.
- Implementar comunicación cliente–servidor en Vercel para recoger eventos y feedback del usuario.

3. Marco teórico

3.1 Tipos de problemas en Machine Learning y taxonomía

En ML, los problemas suelen categorizarse como:

- **Clasificación**: predecir una clase discreta, este caso.
- **Regresión**: predecir un valor continuo.
- **Clustering**: agrupar sin etiquetas.

La taxonomía de aprendizaje incluye:

- **Supervisado**: entrenamiento con etiquetas, en este caso.
- **No supervisado**: sin etiquetas.
- **Por refuerzo**: aprendizaje por recompensa/penalización.

Este proyecto es **aprendizaje supervisado de clasificación multiclase**.

3.2 Clasificación multiclase y softmax

En clasificación multiclase, la red suele producir un vector que se transforma a probabilidades con **softmax**. El entrenamiento se hace típicamente con **entropía cruzada**. La clase con probabilidad máxima es la predicción final.

3.3 CNN y visión por computador

Las **redes convolucionales (CNN)** son adecuadas para imágenes porque explotan la estructura espacial y aprenden filtros jerárquicos (bordes → texturas → patrones). Para estados de un plátano, los rasgos relevantes suelen ser color, manchas, textura y contraste.

3.4 Overfitting/underfitting

- **Underfitting**: modelo demasiado simple; no aprende patrones.
- **Overfitting**: El modelo aprende demasiado el train set y generaliza peor.

Se mitiga con: augmentations, regularización, early stopping, mejor dataset, etc.

3.5 MLOps básico: ciclo de vida del modelo

Dataset → entrenamiento → evaluación → despliegue → observabilidad → feedback → re-entrenamiento.

Este proyecto implementa parte del ciclo completo: despliegue + recogida mínima de feedback para una mejora futura.

3.6 Justificación de tecnologías

- **ONNX Runtime Web**: permite inferencia en navegador, simplificando el despliegue.
- **COCO-SSD (TFJS)**: detector generalista que permite localizar “banana” y recortar automáticamente, mejorando UX.
- **Vercel**: despliegue rápido de estáticos y funciones serverless para comunicación mínima cliente—servidor.

4. Desarrollo del proyecto

4.1 Dataset y preparación de datos

Origen del dataset:Kaggle

Clases: pasado, perfecto, podrido, verde.

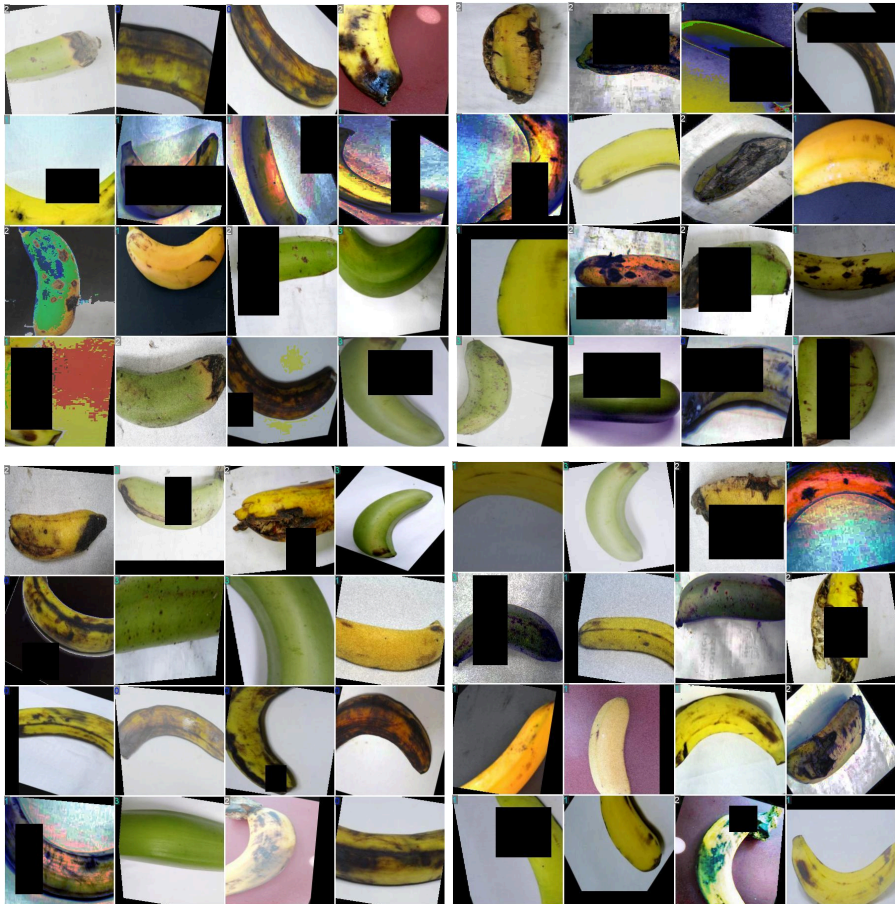
Tamaño total y por clase:la cantidad de fotos por clase son más o menos igual

Split train/val/test:train 87% val 8% y test 4%

Preprocesamiento:

- Redimensionado a 224×224.
- Normalización coherente con el entrenamiento.
- Augmentations típicas: rotaciones, flips, cambios leves de brillo/contraste, recortes.
- Detector: se aplica un detector para localizar el objeto antes de clasificarlo

4.2 EDA (análisis exploratorio)



- Conteo de imágenes por clase.
- Ejemplos representativos por clase (muestras).

Observaciones esperables:

- Clases con apariencias limítrofes: perfecto vs pasado, verde vs casos con sombras/manchas.
- Posibles sesgos: fondos repetidos, iluminación similar, encuadres muy centrados. (por suerte en el dataset hay varios plátanos que son la misma foto pero con filtros aplicados por lo que este dataset está hecho para evitar estos sesgos.)
- “Outputs per training example: 3 Flip: Horizontal, Vertical 90° Rotate: Clockwise, Counter-Clockwise, Upside Down Crop: 0% Minimum Zoom, 20% Maximum Zoom Rotation: Between -15° and +15° Hue: Between -10° and +10° Saturation: Between -10% and +10% Brightness: Between -10% and +10% Exposure: Between -10% and +10% Blur: Up to 1px”

(texto extraído de kaggle, Banana Ripeness Classification

<https://www.kaggle.com/datasets/shahriar26s/banana-ripeness-classification-dataset/data>)

4.3 Modelado: algoritmos y comparativas

1. Baseline (clase mayoritaria)

- Siempre predice la clase más frecuente.
- En tu evaluación: clase mayoritaria = **podrido (388/1123) $\Rightarrow$ 34,55% de acierto.**

2. Modelo clásico

- Logistic Regression o SVM usando **rasgos simples**.
- Sirve para demostrar si con “color” basta o no.

3. Modelo final CNN/ONNX

- Mejor porque aprende textura/manchas además del color.
- Resultado: **98,93% acierto**

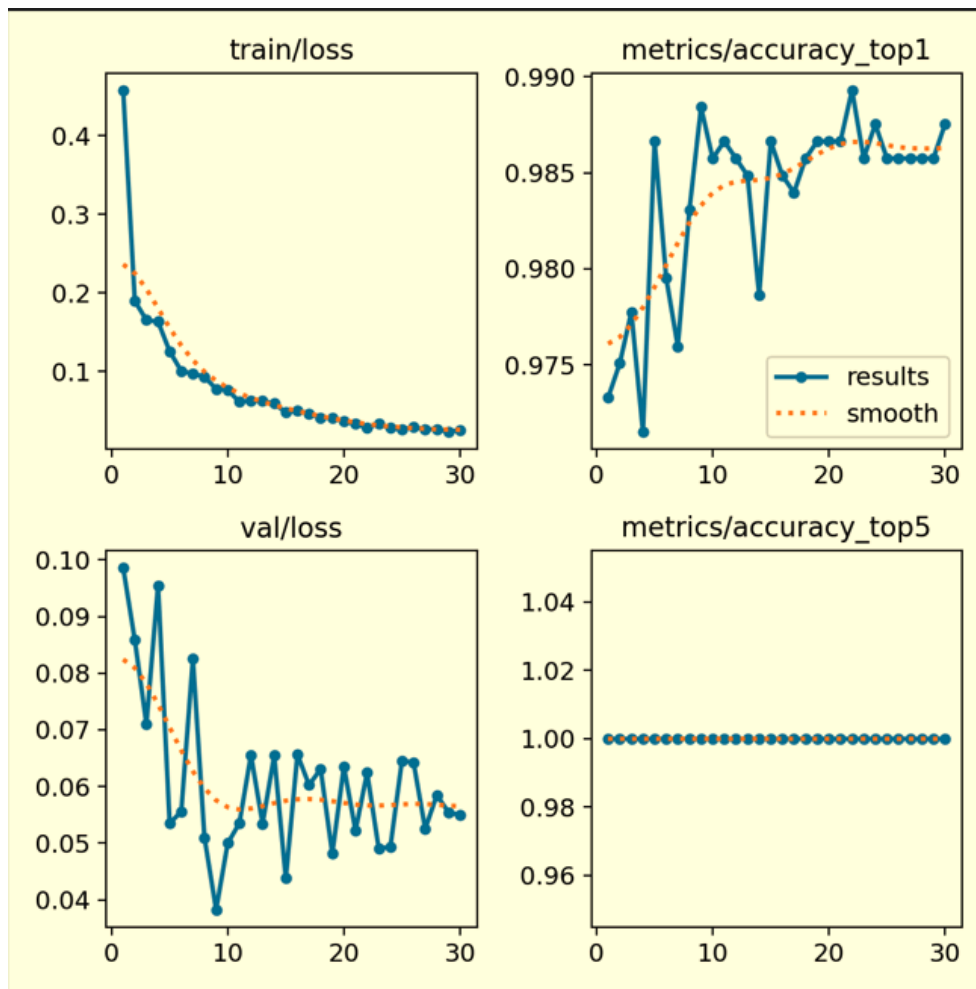
4.4 Entrenamiento, métricas y evaluación

4.4.1 Curvas de entrenamiento

epoch	time	train/loss	metrics/accuracy_top1	metrics/accuracy_top5	val/loss	lr/pg0	lr/pg1	lr/pg2
1	110162	0.45829	0.97329		10.09859	0.000416102	0.000416102	0.000416102
2	233194	0.18991	0.97507		10.08587	0.000805287	0.000805287	0.000805287
3	346419	0.16562	0.97774		10.07102	0.00116697	0.00116697	0.00116697
4	453003	0.16352	0.9715		10.09548	0.00112625	0.00112625	0.00112625
5	545.15	0.12587	0.98664		10.05352	0.001085	0.001085	0.001085
6	639391	0.10073	0.97952		10.05558	0.00104375	0.00104375	0.00104375
7	744308	0.09746	0.97596		10.08253	0.0010025	0.0010025	0.0010025
8	889442	0.09341	0.98308		10.05096	0.00096125	0.00096125	0.00096125
9	1029.16	0.07793	0.98842		10.03827	0.00092	0.00092	0.00092
10	1147.29	0.07682	0.98575		10.05	0.00087875	0.00087875	0.00087875
11	1246.12	0.06142	0.98664		10.05352	0.0008375	0.0008375	0.0008375
12	1355.91	0.06318	0.98575		10.06555	0.00079625	0.00079625	0.00079625
13	1466.45	0.06245	0.98486		10.05345	0.000755	0.000755	0.000755
14	1575.03	0.05938	0.97863		10.06545	0.00071375	0.00071375	0.00071375
15	1685.75	0.04807	0.98664		10.04381	0.0006725	0.0006725	0.0006725
16	1793.65	0.05008	0.98486		10.06564	0.00063125	0.00063125	0.00063125
17	1903.71	0.04679	0.98397		10.06028	0.00059	0.00059	0.00059
18	2015.26	0.04161	0.98575		10.06308	0.00054875	0.00054875	0.00054875
19	2125.3	0.04087	0.98664		10.04816	0.0005075	0.0005075	0.0005075
20	2232.22	0.03658	0.98664		10.0635	0.00046625	0.00046625	0.00046625
21	2343.08	0.03346	0.98664		10.05222	0.000425	0.000425	0.000425
22	2453.05	0.02861	0.98931		10.06245	0.00038375	0.00038375	0.00038375
23	2564.13	0.03335	0.98575		10.04906	0.0003425	0.0003425	0.0003425
24	2676.19	0.02816	0.98753		10.04935	0.00030125	0.00030125	0.00030125
25	2787.12	0.02628	0.98575		10.06458	0.00026	0.00026	0.00026
26	2895.59	0.02922	0.98575		10.0642	0.00021875	0.00021875	0.00021875
27	3007.55	0.02676	0.98575		10.05253	0.0001775	0.0001775	0.0001775
28	3120.11	0.02692	0.98575		10.05848	0.00013625	0.00013625	0.00013625
29	3232.17	0.0232	0.98575		10.05547	9.5e-05	9.5e-05	9.5e-05
30	3340.96	0.02583	0.98753		10.05494	0.05375	0.05375	0.05375

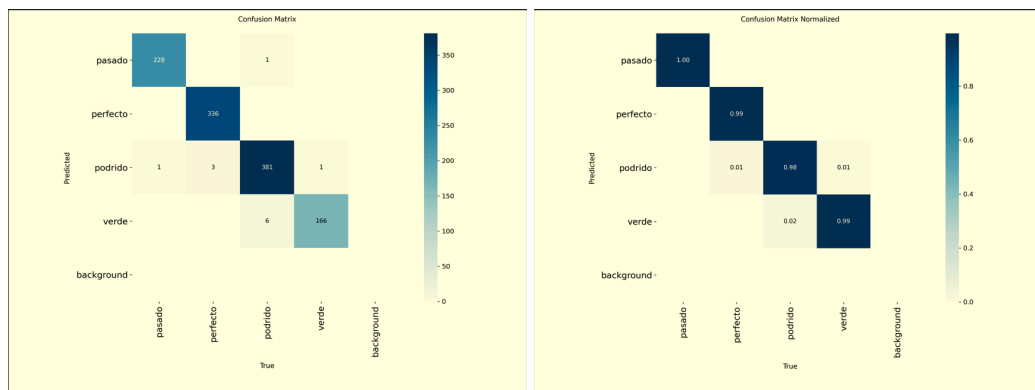
- **Top-1 accuracy (máxima):** 0.98931 (epoch 22)
- **Top-1 accuracy (final):** 0.98753 (epoch 30)
- **Top-5 accuracy:** 1.0 durante todo el entrenamiento
- **Train loss (final):** 0.02583
- **Val loss (mínima):** 0.03827 (epoch 9)
- **Val loss (final):** 0.05494

Interpretación (overfitting/underfitting):



El train loss desciende de forma sostenida mientras la val loss mejora rápido y luego oscila. Esto sugiere un entrenamiento saludable sin underfitting; el tramo final train/val indica un grado de sobreajuste **ligero** esperado, pero las métricas globales siguen siendo muy altas.

4.4.2 Matriz de confusión y métricas por clase



Según la matriz de confusión (1123 muestras):

- **Accuracy global:** 0.9893 (98.93%)
- **Macro-F1:** 0.9888
- **Weighted-F1:** 0.9893

Por clase (Precision / Recall / F1):

- pasado: 0.9956 / 0.9956 / 0.9956
- perfecto: 1.0000 / 0.9912 / 0.9956
- podrido: 0.9870 / 0.9820 / 0.9845
- verde: 0.9651 / 0.9940 / 0.9794
-

4.4.3 Análisis de errores

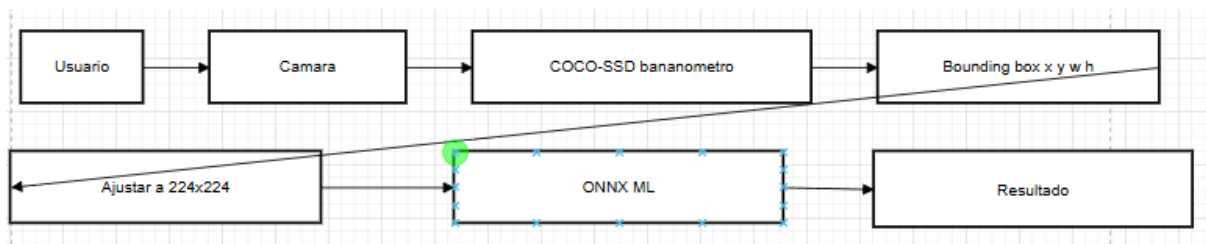
Los errores son escasos y se concentran en clases visualmente cercanas:

- Confusiones puntuales **podrido** ↔ **verde** (posibles sombras, reflejos o zonas verdes residuales en plátanos deteriorados).
- Algunos casos **perfecto** → **podrido** (manchas oscuras o deterioro localizado).

Estos fallos son relevantes porque un falso negativo/positivo puede afectar a la decisión del usuario. Por ello en la aplicación cuando el grado de confianza es bajo , la aplicación te avisa ya que es más probable que se esté equivocando, los errores tienen que ver con los plátanos podridos directa o indirectamente.

4.5 Arquitectura y despliegue

4.5.1 Arquitectura general



4.5.2 Estructura de proyecto

- public/ (front)
- api/ (funciones serverless)
- package.json (ESM)

4.5.3 Decisiones de ingeniería

Se detectó un problema de UX: el plátano debía ocupar mucho espacio en el recorte central para ser clasificado bien. Se resolvió con un enfoque robusto: detector + recorte automático, similar a las detecciones de cara dadas en clase.

4.6 Comunicación cliente–servidor y ciclo de feedback

Se implementó comunicación usando /api/log:

- Eventos: connected, detected_perfect, user_feedback.
- Payload: userId , timestamp, etiqueta confirmada por el usuario, predicción del modelo y confianza.

Ciclo de feedback:

- Los eventos de feedback del usuario se guardarán en una base de datos para luego reentrenar el modelo con ellos.
- El re-entrenamiento futuro usará:
 - ejemplos donde el modelo falla,
 - ejemplos “difíciles” (baja confianza),
 - feedback del usuario.

Privacidad:

- No se almacena vídeo continuo; el snapshot es opcional y solo bajo acción del usuario. En el caso de que el usuario no lo consienta no se usarán sus datos para el reentrenamiento.

4.7 Incidencias y resolución (ingeniería)

Durante el desarrollo se identificaron y resolvieron problemas reales:

1. **404 en Vercel** por estructura de carpetas / Root Directory incorrecto.
2. Confusión de ramas GitHub/Vercel (desplegaba master cuando el código estaba en otra rama).
3. Error de configuración en Vercel por runtime/ESM; se solucionó con package.json (type: module) y export default en funciones.
4. **Error “protobuf parsing failed”** al cargar ONNX: se debía a archivo ONNX vacío o no servido como binario; se resolvió exportando el ONNX correctamente.
5. **Exportación ONNX y compatibilidad Python**: problemas con Python 3.12 y dependencias; se resolvió usando Python 3.11 para exportación.
6. **UX insuficiente (recuadro fijo)**: el modelo necesitaba plátano grande; se introdujo detección banana + crop dinámico.

Esta sección demuestra iteración técnica y control del ciclo “desarrollo → fallo → diagnóstico → solución”.

5. Conclusiones

- El modelo alcanza rendimiento muy alto ($\approx 98.9\%$ acierto), con fallos concentrados en casos específicos (plátanos podridos), después de intentar engañar a la IA con objetos similares a plátanos pero que no lo son, mi teoría es que si detecta algo similar a un plátano pero alguna de sus características es muy extraña lo cataloga como plátano podrido.
- La aplicación web es funcional y presenta predicción en tiempo real.
- La robustez en entorno real depende de la detección del objeto, condiciones de iluminación, diversidad del dataset y calidad de la cámara.
- Próximos pasos: persistencia de feedback en base de datos, detector propio entrenado específicamente, ampliación del dataset y evaluación con datos reales de cámara.

6. Evaluación crítica

Por qué ML es esencial

aunque en el mundo real hay demasiados factores que pueden alterar los resultados como iluminación, ángulo, calidad de la cámara, distancia del objetivo ... No podemos negar la utilidad de un LM para reconocer patrones, además en el caso de no acertar puede cuantificarse la confianza y aprender del feedback.

Evaluación crítica de la solución respecto a requisitos del usuario

Fortalezas:

- Respuesta en tiempo real.
- estructura clara (overlay y mensajes).
- Buena precisión en test.

Limitaciones:

- Detector COCO-SSD puede fallar en algunos encuadres y móviles.

- Dependencia de iluminación/fondo.

Roadmap:

- Persistencia en BD para feedback.
- Re-entrenamiento periódico con casos reales.
- Monitorización(nuevos fondos, nuevas cámaras).

7. Bibliografía

IBM *¿Qué son las redes neuronales convolucionales?* IBM Think.

TensorFlow *Aprendizaje automático para desarrolladores de JavaScript (TensorFlow.js).*
TensorFlow (ES).

Ultralytics *ONNX para modelos YOLO (integración ONNX).* Documentación de Ultralytics (ES).